



UNIVERSIDAD NACIONAL DE ROSARIO
FACULTAD DE CIENCIAS EXACTAS, INGENIERÍA Y
AGRIMENSURA

Tecnicatura Universitaria en Inteligencia Artificial
Procesamiento de imágenes I – IA 4.4

TRABAJO PRÁCTICO N° 1 – Año 2026 – 2° Semestre

Profesores: Gonzalo Sad, Juan Manuel Calle, Joaquín Allione

Integrantes y legajos:

Agustín Torres – 42768902

Aldana Desiré Sánchez – 41904105

Facundo Magliaro – M-7386/5

Link repositorio: https://github.com/A6u5/TP1_PDI_Torres_Sanchez_Magliaro

ÍNDICE

1. INTRODUCCIÓN

2. DESARROLLO

2.1 PROBLEMA 1: ECUALIZACIÓN LOCAL DE HISTOGRAMA

2.1.1 Implementación de la ecualización local (punto a)

2.1.2 Detalles ocultos en la imagen (punto b)

2.1.3 Influencia del tamaño de la ventana (punto c)

2.2 PROBLEMA 2: VALIDACIÓN DE PLANILLAS DE CALIFICACIONES

2.2.1 Validación de los campos de la planilla (punto a)

2.2.2 imagen de alumnos no aprobados (punto b)

2.2.3 Archivo CSV de resultados (punto c)

2.2.4 Aplicación cíclica sobre las cuatro planillas (punto d)

3. CONCLUSIONES

ANEXO

Valores máximos de las sumas por columnas y filas

1. INTRODUCCIÓN

El procesamiento digital de imágenes permite mejorar la calidad visual de una imagen y extraer información que a simple vista no resulta evidente, así como automatizar tareas de análisis que de otro modo requerirían inspección manual. En este trabajo se abordan dos problemas de esa naturaleza.

En el Problema 1 se estudia la ecualización local de histograma. La ecualización global redistribuye los niveles de intensidad con un único histograma calculado sobre toda la imagen, por lo que queda dominada por las zonas de mayor extensión y se pierde la información local. La versión local calcula un histograma distinto para cada píxel a partir de una ventana de tamaño $M \times N$ centrada en él. Se implementa el algoritmo y se lo aplica a una imagen con detalles ocultos en distintas zonas para identificarlos, analizando además la influencia del tamaño de la ventana, con valores entre 3×3 y 121×121 píxeles.

En el Problema 2 se desarrolla un script que verifica de forma automática planillas de calificaciones en formato imagen. A partir de la detección de las líneas que delimitan la tabla y de la segmentación de los caracteres de cada campo mediante componentes conectadas, se comprueba si los datos de cada registro cumplen las restricciones definidas para el legajo, el nombre y apellido, las notas de los parciales y la condición final. Como resultado, el algoritmo informa por pantalla qué campos son correctos (OK) y cuáles no (MAL), genera una imagen con los alumnos que no aprobaron y almacena las validaciones en un archivo CSV por planilla.

Ambos problemas se resuelven en Python con las librerías OpenCV, NumPy, Pillow y Matplotlib. El código se organiza en los módulos problema1.py y problema2.py, y el script main.py ejecuta ambos problemas de forma secuencial y guarda todas las salidas en disco.

2. DESARROLLO

2.1 PROBLEMA 1: ECUALIZACIÓN LOCAL DE HISTOGRAMA

2.1.1 Implementación de la ecualización local (punto a)

La ecualización global calcula un único histograma con todos los píxeles de la imagen. En la imagen analizada, el fondo claro ocupa la mayor parte de la superficie y domina ese histograma, por lo que las diferencias de intensidad dentro de los cuadrados oscuros no se

logran distinguir. La ecualización local resuelve esto calculando, para cada píxel, un histograma solo con los píxeles de una ventana $M \times N$ centrada en él.

La función `ecualizacion_local_hist(img, M, N=None)`, definida en `problema1.py`, recibe la imagen en escala de grises y el tamaño de la ventana (si no se indica N , la ventana es cuadrada, $N = M$). Sigue estos pasos:

1. Agrega un borde de replicación con `cv2.copyMakeBorder(..., cv2.BORDER_REPLICATE)`, de $M//2$ píxeles arriba, $M-1-M//2$ abajo, $N//2$ a la izquierda y $N-1-N//2$ a la derecha. De este modo la ventana puede centrarse también en los píxeles del borde de la imagen, y el cálculo es válido para tamaños pares e impares.
2. Para cada píxel (i, j) toma la ventana $M \times N$ correspondiente de la imagen con borde y calcula su histograma con `np.bincount(ventana.ravel(), minlength=256)` y su función de distribución acumulada (CDF) con `np.cumsum`.
3. Calcula el nuevo nivel del píxel central con la transformación de ecualización $s = \text{round}((L-1) \cdot \text{CDF}(r) / (M \cdot N))$, con $L = 256$ y r el nivel original del píxel, y lo guarda en la imagen de salida.

En otras palabras, el nuevo valor indica qué fracción de la ventana es igual o más oscura que el píxel central. Un píxel más claro que su entorno pasa a valores altos y uno más oscuro pasa a valores bajos, de modo que la imagen resalta las diferencias locales aunque en la imagen original distan apenas uno o dos niveles de gris.

La lectura de la imagen se realiza con Pillow y se convierte a escala de grises `uint8` (`Image.open(...).convert("L")`). La función `resolver_problema1` calcula una única vez el resultado de cada tamaño de ventana (el procesamiento es lento por recorrer la imagen píxel a píxel) y lo reutiliza en los puntos b y c.

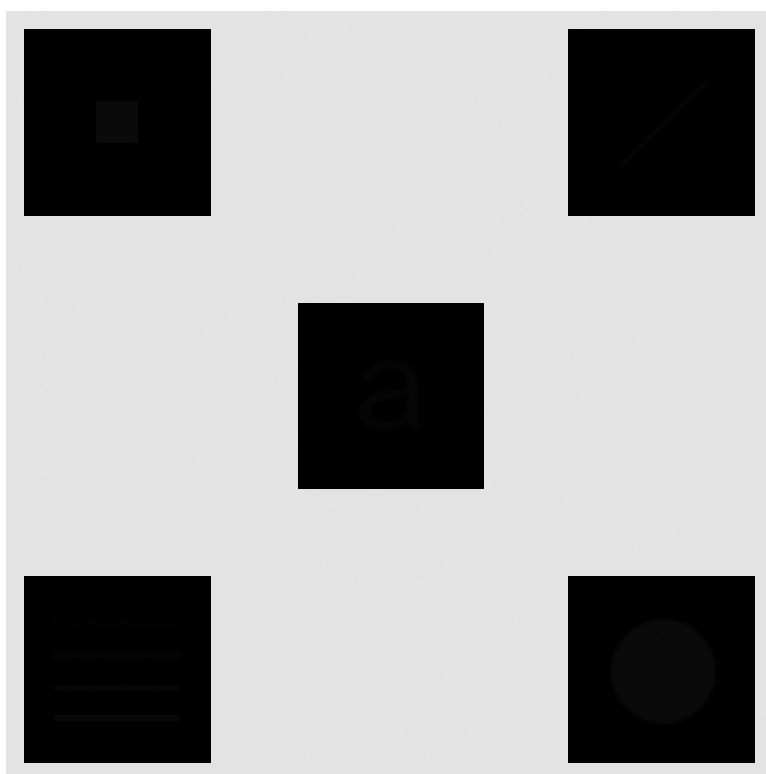


Figura 1: imagen original. Los detalles ocultos no se distinguen a simple vista.

2.1.2 Detalles ocultos en la imagen (punto b)

Se utilizó una ventana de 7x7 píxeles (constante `M_ELEGIDA = 7` en `problema1.py`). La Figura 2 compara la imagen original con el resultado, y los detalles que se revelan en cada zona son:

Zona	Detalle oculto
Cuadrado superior izquierdo	Un cuadrado pequeño centrado
Cuadrado superior derecho	Una línea diagonal (de abajo a la izquierda hacia arriba a la derecha)
Cuadrado central	La letra "a" minúscula
Cuadrado inferior izquierdo	Un conjunto de líneas horizontales paralelas (cuatro)
Cuadrado inferior derecho	Un círculo
imagen completa	Puntos distribuidos al azar a lo largo de todo el fondo (incluso pueden verse dentro del pequeño cuadrado, en el círculo, en las líneas dibujadas y en la letra 'a', como si los grandes cuadrados negros de las esquinas y del centro estuvieran 'por encima' del fondo y sus representaciones internas fueran recortes)

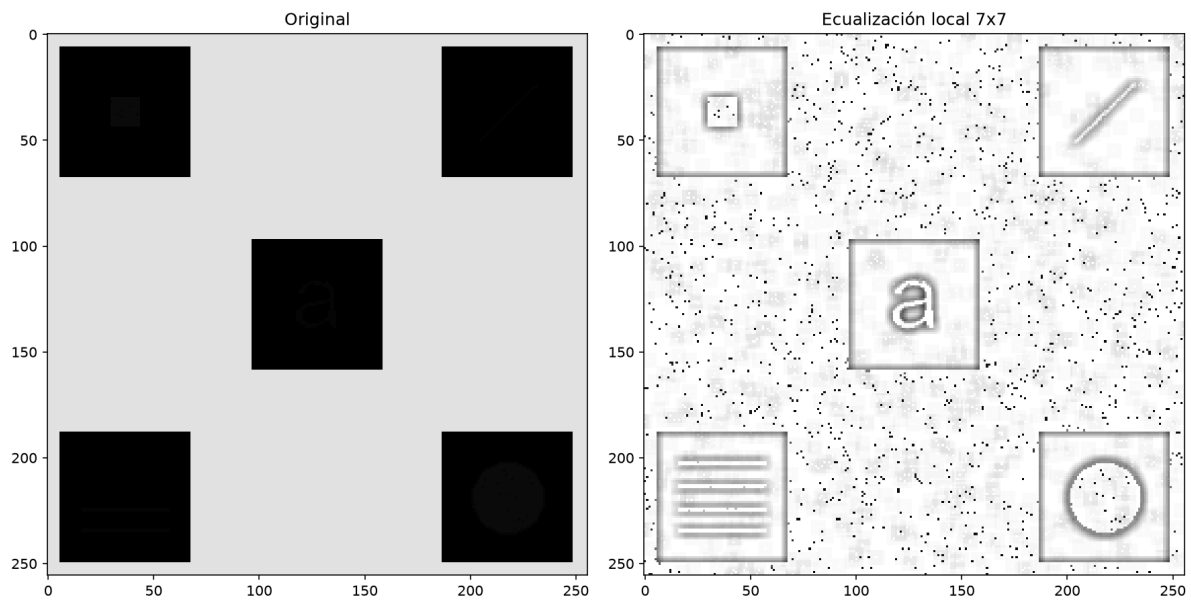


Figura 2: imagen original (izquierda) y resultado de la ecualización local con ventana de 7x7 (derecha).

2.1.3 Influencia del tamaño de la ventana (punto c)

Se aplicó la función con ventanas cuadradas de 3, 7, 15, 31, 61 y 121 píxeles (lista TAMANIOS en problema1.py).

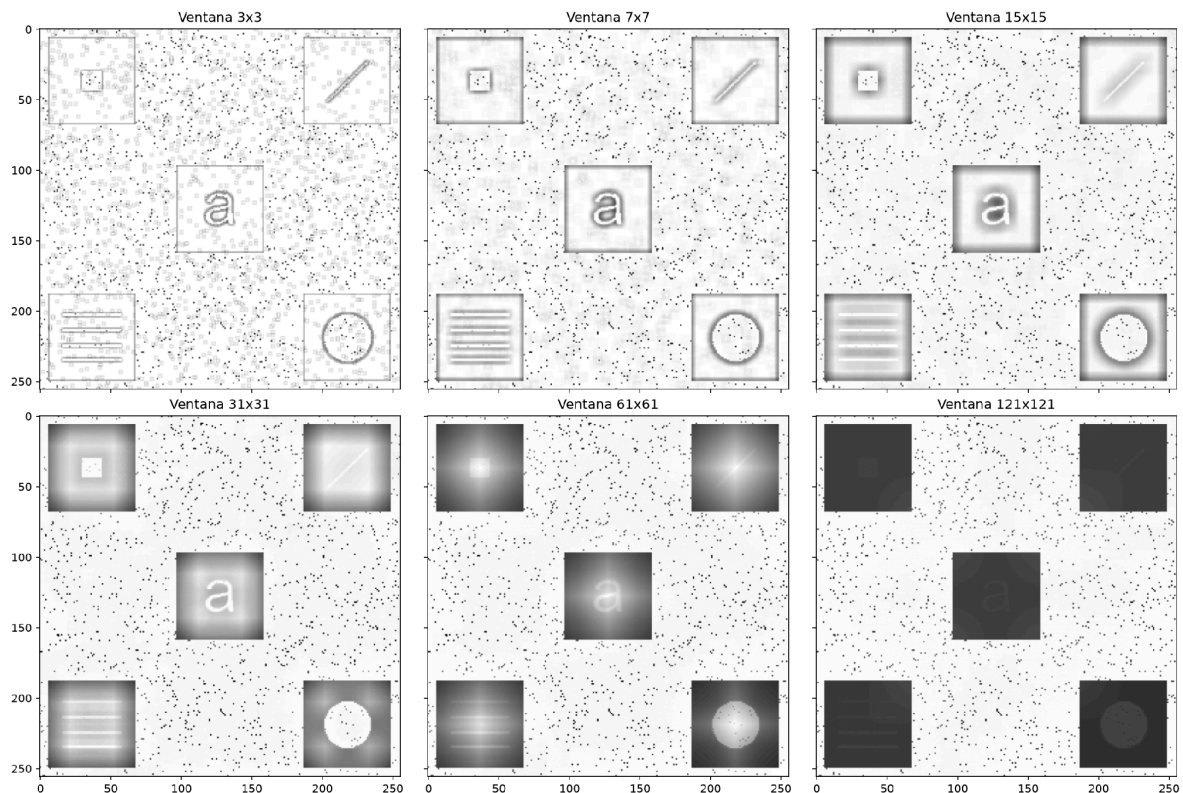


Figura 3: Resultado de la ecualización local para ventanas de 3x3, 7x7, 15x15, 31x31, 61x61 y 121x121.

Ventana	Observaciones
3x3	Los cinco detalles se distinguen, pero solo como contornos muy finos y de bajo contraste. Es la que más puntos oscuros muestra en el fondo, y los bordes de las figuras se ven irregulares.
7x7	Los contornos son claros y nítidos, y las figuras están bien delimitadas. Los halos alrededor de los cuadrados son leves. Es el mejor equilibrio entre visibilidad de los detalles y fidelidad de la forma.
15x15	Los detalles siguen siendo claros y las figuras se ven más definidas. Comienzan a aparecer halos más marcados, es decir, degradados de gris cerca de los bordes de los cuadrados grandes.
31x31	Las figuras se ven como formas claras sobre un fondo gris, y el círculo aparece como un disco. Los degradados alrededor de los bordes son muy marcados y el interior de los cuadrados deja de ser uniforme.
61x61	Aparecen artefactos importantes: manchas con forma de cruz o estrella, y las figuras se vuelven borrosas. La letra "a" y las líneas se distinguen con dificultad.
121x121	Los cuadrados quedan casi uniformes y las figuras apenas se insinúan con un contraste muy bajo. El resultado se parece al de la ecualización global.

Explicación. El resultado depende de cuánto contenido "ve" la ventana en cada posición:

- Ventanas chicas (3 a 15): la ventana es del tamaño de los detalles o menor. Cuando está sobre una figura, ve el trazo y el fondo oscuro inmediato, así que el contraste local se amplifica y el detalle se revela con precisión. El costo es una mayor sensibilidad al ruido, sobre todo con 3x3.
- Ventanas medianas (31): la ventana ya abarca el detalle completo y parte del entorno, y cerca de los bordes de los cuadrados incluye píxeles claros del fondo exterior. Eso altera el histograma local y produce los degradados observados.
- Ventanas grandes (61 y 121): la ventana incluye casi todo el cuadrado y buena parte del fondo. El histograma queda dominado por la diferencia entre fondo claro y cuadrado oscuro, no por los detalles internos, que se vuelven irrelevantes estadísticamente. Los artefactos en forma de cruz se deben a que la ventana cuadrada cruza los bordes de los cuadrados. Al crecer la ventana, el método tiende a la ecualización global.

Sobre los puntos oscuros del fondo. Los puntos negros dispersos en el fondo aparecen con todos los tamaños de ventana, incluso con 121x121, donde el resto del contraste desaparece. En la imagen original el fondo toma únicamente los valores 226, 227 y 228, por lo que existen píxeles aislados apenas más oscuros que su entorno (226 frente a 227). Al ser los más oscuros

de su vecindario, la transformación los lleva a valores cercanos a 0, y por eso se ven aunque no se noten en la imagen original. Con ventanas pequeñas el efecto es mayor porque hay menos píxeles con los que compararse.

2.2 PROBLEMA 2: VALIDACIÓN DE PLANILLAS DE CALIFICACIONES

El problema consiste en analizar planillas de calificaciones en formato imagen (archivos `grade_sheet_<id>.png`). Cada registro ocupa una fila y tiene dos campos de datos personales (Legajo, Nombre y Apellido) y cuatro de calificaciones (Parcial 1, Parcial 2, Parcial 3 y Condición Final). Las restricciones a verificar son:

- Legajo: exactamente 8 caracteres, formando una única palabra.
- Nombre y Apellido: al menos dos palabras y no más de 12 caracteres en total.
- Parcial 1, 2 y 3: 1 o 2 caracteres consecutivos, sin espacios entre ellos.
- Condición Final: un único carácter.

Solo se verifica que cada dato cumpla las restricciones del campo, no que la información registrada sea válida. Para ilustrar el procedimiento se usa como ejemplo la imagen `grade_sheet_1.png`. La solución está en `problema2.py`, y `main.py` aplica el algoritmo a las cuatro planillas.

2.2.1 Validación de los campos de la planilla (punto a)

La validación está implementada en `validar_planilla(img)`, que recibe únicamente la imagen de la planilla en escala de grises (`cv2.imread(..., cv2.IMREAD_GRAYSCALE)`). Se realiza en los siguientes pasos.

Paso 1 – Binarización. Se umbraliza la imagen con `img_th = img < 128`, de modo que los píxeles oscuros (texto y líneas de la tabla) quedan en `True` y el fondo en `False`.

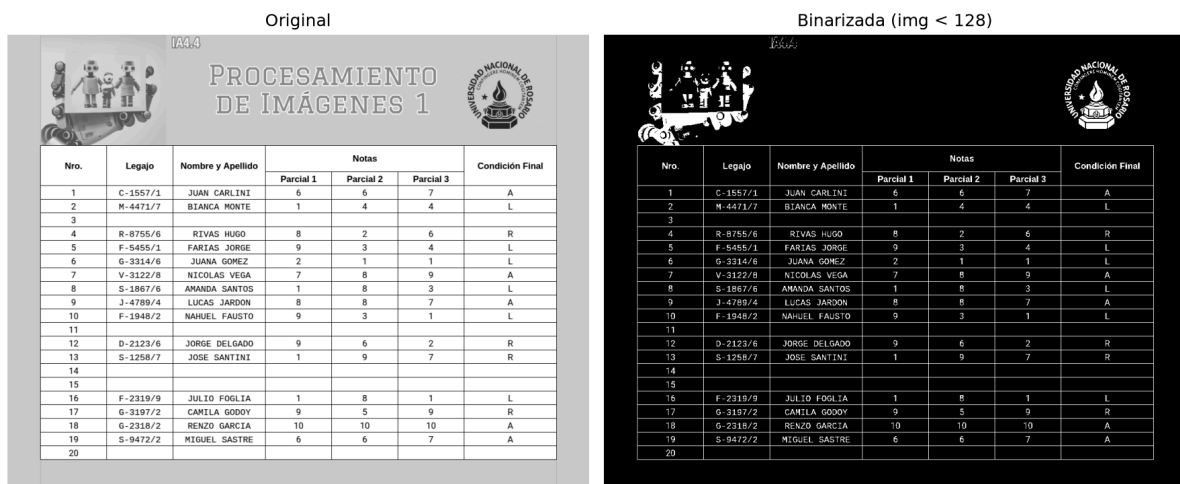


Figura 4: Planilla grade_sheet_1.png original (izquierda) y con el umbral aplicado (derecha).

Paso 2 – Sumas por filas y columnas. Para detectar la estructura de la tabla se suman los píxeles oscuros de cada columna, $\text{img_cols} = \text{np.sum}(\text{img_th}, \text{axis}=0)$, y de cada fila, $\text{img_rows} = \text{np.sum}(\text{img_th}, \text{axis}=1)$. El primer vector tiene tantos elementos como columnas tiene la imagen, y cada posición contiene la cantidad de píxeles oscuros de esa columna; lo mismo ocurre con las filas. Las líneas verticales y horizontales de la tabla atraviesan toda la planilla, por lo que producen picos muy superiores al resto del contenido.

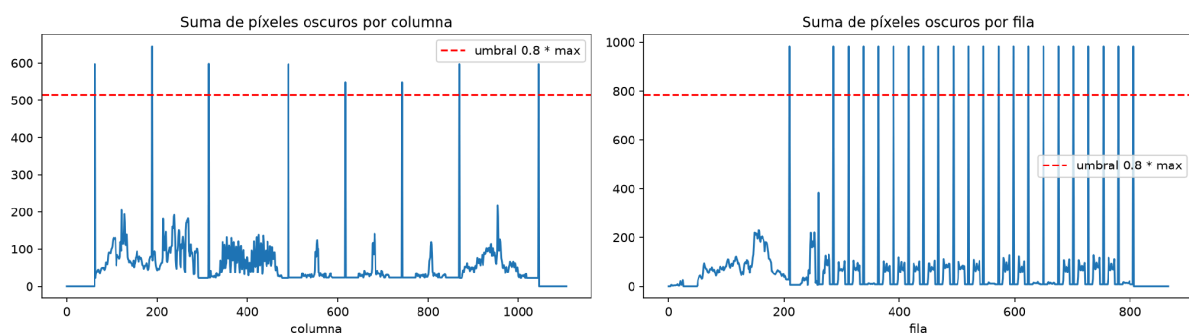


Figura 5: Sumas de píxeles oscuros por columna (izquierda) y por fila (derecha). Los picos corresponden a las líneas de la tabla; la línea punteada es el umbral.

Paso 3 – Umbral de líneas. Para decidir si una columna o fila corresponde a una línea se define un umbral para cada dirección a partir del máximo de las sumas: $\text{th_col} = 0.8 * \text{max}(\text{img_cols})$ y $\text{th_row} = 0.8 * \text{max}(\text{img_rows})$. Las posiciones que superan el umbral ($\text{img_cols} > \text{th_col}$, $\text{img_rows} > \text{th_row}$) se consideran líneas. Los valores máximos obtenidos en cada imagen se listan en el Anexo 02.

Paso 4 – Agrupación de posiciones consecutivas. Como una línea puede tener más de un píxel de espesor, la función auxiliar `agrupar_consecutivos(indices)` reúne los índices

consecutivos en grupos (inicio, fin) y centros_de_grupos(grupos) toma el centro de cada uno como coordenada de la línea. Se obtienen así las listas x (líneas verticales) e y (líneas horizontales). En las cuatro planillas se detectan 8 líneas verticales (7 columnas) y 22 horizontales; en estas imágenes cada línea tiene 1 píxel de espesor, pero la función queda preparada para espesores mayores.

Paso 5 – Recorrido de celdas. Dos líneas horizontales consecutivas ($y[i]$, $y[i+1]$) y dos verticales consecutivas ($x[j]$, $x[j+1]$) delimitan una celda, cuyo recorte es $\text{img_th}[y[i]+1 : y[i+1], x[j]+1 : x[j+1]]$ (se descuenta la línea misma). Se recorren los registros fila por fila, a partir de la fila 1 ($\text{fila_inicio_datos} = 1$) para omitir el encabezado, y las columnas desde $j = 1$, para omitir la columna "Nro.", que no forma parte de las validaciones. Cada columna j se asocia a su campo según la lista COLUMNAS (Legajo, Nombre y Apellido, Parcial 1, Parcial 2, Parcial 3, Condición Final). Cada planilla tiene 20 registros.

Paso 6 – Caracteres de cada celda. Para analizar el contenido de una celda, la función `componentes_validas(celda, area_min)` emplea `cv2.connectedComponentsWithStats` con `connectivity=8`, que identifica los grupos de píxeles conectados entre sí. Cada componente conexa se interpreta como un carácter, y la función devuelve su posición (x, y), ancho, alto y área, descartando la etiqueta 0 (fondo). Para eliminar componentes espurias producidas por ruido o por restos de las líneas de la tabla se conservan solo las de área mayor o igual a $\text{area_min} = 4$. Este valor se determinó experimentalmente, teniendo en cuenta que algunos caracteres válidos, como el guion, tienen un área considerablemente menor que las letras o los números.

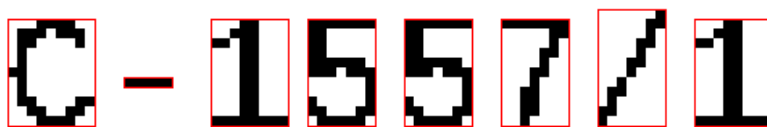


Figura 7: Celda correspondiente a la primera fila y la primera columna de datos de `grade_sheet_1.png`, con las componentes conectadas (caracteres) detectadas; en este ejemplo se detectan 8.

Paso 7 – Palabras. La función `cantidad_palabras(componentes, espacio_min=10)` ordena las componentes por su posición horizontal y calcula la distancia entre el final de una componente ($x + w$) y el comienzo de la siguiente. Cuando esa distancia es mayor o igual a

espacio_min = 10 píxeles se considera que hay un espacio entre palabras y se incrementa el contador (se parte de una palabra si existe al menos una componente).

Paso 8 – Reglas por campo. Con la cantidad de componentes (caracteres) y de palabras se aplican las restricciones del enunciado según la columna:

Campo	Condición para "OK"
Legajo	8 caracteres y 1 palabra
Nombre y Apellido	12 caracteres como máximo y 2 palabras o más
Parcial 1, 2 y 3	1 o 2 caracteres y 1 palabra (sin espacios)
Condición Final	Exactamente 1 carácter

Si la celda no cumple la regla, el resultado es "MAL". Cada registro se devuelve como un diccionario con su número ("nro"), el estado de cada campo ("campos": OK/MAL), los límites de cada celda ("celdas": (y0, y1, x0, x1)) y un indicador "correcto" que es True si todos los campos son OK. Los límites se conservan porque el punto b los reutiliza para recortar el nombre del alumno y leer su condición final.

Salida por pantalla. La función mostrar_resultados(registros) imprime, por cada registro, el estado de sus campos con el formato pedido ("Registro 1:", "Legajo: OK", etc.).

```
=====
PROBLEMA 2 - PLANILLA 1 (grade_sheet_1.png)
=====
Registro 1:
Legajo: OK
Nombre y Apellido: OK
Parcial 1: OK
Parcial 2: OK
Parcial 3: OK
Condición Final: OK

Registro 2:
Legajo: OK
Nombre y Apellido: OK
Parcial 1: OK
Parcial 2: OK
Parcial 3: OK
Condición Final: OK
```

Figura 8: Salida por pantalla de la validación de la planilla `grade_sheet_1.png`.

2.2.2 imagen de alumnos no aprobados (punto b)

La función `generar_imagen_no_aprobados(img, registros)` produce una única imagen con los alumnos que no aprobaron (Condición Final "L" o "R"), considerando solamente los registros cargados correctamente (`registro["correcto"] == True`), como pide el enunciado.

Clasificación de la condición final. Para cada registro correcto se recorta la celda de Condición Final de la imagen binaria y se la clasifica con `clasificar_condicion(celda)`. Como las letras posibles en este campo que interesan son L, R y las de aprobado (por ejemplo "A"), se las distingue por su forma sin usar reconocimiento de caracteres:

1. Se obtiene la única componente conectada de la celda (`componentes_validas`) y se recorta la letra a su bounding box. Si no hay exactamente una componente, la celda no se reconoce (None).
2. `contar_huecos(letra)` invierte la letra (el fondo pasa a ser el objeto) y cuenta sus componentes conectadas con `connectivity=4`. Se resta 2 al total (la etiqueta 0 y el fondo exterior), con lo que se obtiene la cantidad de huecos encerrados. La "L" no tiene huecos, por lo que se la devuelve como "L".

3. Si la letra tiene al menos un hueco, `columna_izquierda_llena(letra, tolerancia=0.9)` verifica si el trazo vertical izquierdo ocupa de arriba a abajo al menos el 90 % de la altura. Si es así se trata de una "R" (trazo izquierdo vertical); en caso contrario (trazo diagonal, como en la "A") se devuelve None, que representa a un alumno aprobado u otra letra no considerada.

Armado de la imagen. Para cada alumno no aprobado, `armar_fila(img, celda_nombre, condicion)` recorta la celda de Nombre y Apellido de la imagen original (el "crop" pedido), le agrega a la izquierda una franja de color con la letra de la condición y un borde del mismo color alrededor. Los colores se definen en `COLOR_CONDICION`: naranja (BGR (0, 165, 255)) para "R" (recupera) y rojo ((0, 0, 255)) para "L" (libre). Las filas se agrupan en dos bloques, cada uno con un encabezado generado por `armar_encabezado ("RECUPERAN (R)" y "LIBRES (L)"),` y se apilan con `np.vstack`. Si no hay ningún alumno para mostrar, la función devuelve None y no se genera imagen.

Resultados. `main.py` guarda la imagen como `imagenes_problema2/no_aprobados_<id>.png`:

Planilla	Registros correctos	Recuperan (R)	Libres (L)	imagen generada
1	15 de 20	4	6	no_aprobados_1.png
2	3 de 20	0	3	no_aprobados_2.png
3	1 de 20	0	0	(no se genera)
4	6 de 20	2	2	no_aprobados_4.png

En la planilla 3 el único registro cargado correctamente corresponde a un alumno aprobado, por lo que no hay ningún no aprobado para informar y no se genera imagen.

RECUPERAN (R)	
R	RIVAS HUGO
R	JORGE DELGADO
R	JOSE SANTINI
R	CAMILA GODOY
LIBRES (L)	
L	BIANCA MONTE
L	FARIAS JORGE
L	JUANA GOMEZ
L	AMANDA SANTOS
L	NAHUEL FAUSTO
L	JULIO FOGLIA

Figura 9: Alumnos no aprobados de grade_sheet_1.png: bloque de los que recuperan (R, naranja) y bloque de los libres (L, rojo).

2.2.3 Archivo CSV de resultados (punto c)

La función `exportar_csv(registros, ruta)` guarda los resultados de la validación de cada planilla en `csv/resultados_<id>.csv`. Cada fila corresponde a un registro: la primera columna es el ID, que sigue el orden de los registros de la planilla (1 a 20), y luego una columna por campo en el orden pedido (Legajo, Nombre y Apellido, Parcial 1, Parcial 2, Parcial 3, Condición Final), con los únicos valores posibles "OK" o "MAL". El archivo se escribe con `newline=""` (recomendado por el módulo `csv` para no duplicar saltos de línea en Windows) y con codificación `utf-8-sig` para que Excel muestre correctamente la tilde de "Condición Final". Un fragmento de `csv/resultados_2.csv`:

```
ID,Legajo,Nombre y Apellido,Parcial 1,Parcial 2,Parcial 3,Condición
Final
1,OK,MAL,OK,OK,OK,OK
2,OK,OK,OK,OK,OK,OK
3,MAL,MAL,OK,OK,OK,OK
```

2.2.4 Aplicación cíclica sobre las cuatro planillas (punto d)

main.py busca en datos/planillas todos los archivos grade_sheet_<id>.png con un identificador numérico (descartando grade_sheet_empty.png) y, para cada uno, en orden: lee la imagen, ejecuta validar_planilla, imprime los resultados con mostrar_resultados, informa la cantidad de registros cargados correctamente, exporta el CSV y genera la imagen de no aprobados. Los resultados fueron:

Planilla	Registros correctos (todos los campos OK)	CSV	imagen de no aprobados
grade_sheet_1	15 de 20	csv/resultados_1.csv	no_aprobados_1.png
grade_sheet_2	3 de 20	csv/resultados_2.csv	no_aprobados_2.png
grade_sheet_3	1 de 20	csv/resultados_3.csv	(ninguna)
grade_sheet_4	6 de 20	csv/resultados_4.csv	no_aprobados_4.png

El detalle campo por campo de cada registro se encuentra en los CSV mencionados.

3. CONCLUSIONES

Problema 1

- La ecualización local revela detalles que la global no puede mostrar, porque analiza cada zona con su propio histograma y no con el de toda la imagen. En la imagen estudiada, los detalles difieren del fondo local en apenas uno o dos niveles de gris.
- Los objetos ocultos son: un cuadrado pequeño, una línea diagonal, la letra "a", un conjunto de líneas horizontales, un círculo y puntos al azar.
- El tamaño de la ventana es el parámetro clave. Debe ser del orden del tamaño de los detalles que se buscan, o algo menor: si es demasiado pequeña, amplifica el ruido; si es demasiado grande, el resultado tiende a la ecualización global y se pierde la información local. Para esta imagen se eligió la ventana de 7x7, que muestra los detalles con contornos claros y sin los halos ni artefactos de las ventanas mayores.

Problema 2

- La estructura de la planilla puede recuperarse de forma robusta con operaciones simples: binarización, sumas por filas y columnas y un umbral relativo al máximo (0.8

· máximo), lo que evita fijar valores arbitrarios para cada imagen. La agrupación de índices consecutivos permite además tratar líneas de más de un píxel de espesor.

- Las componentes conectadas permiten contar caracteres y, mediante la distancia horizontal entre ellas, palabras, sin necesidad de reconocer el texto. El filtrado por área mínima ($\text{area_min} = 4$) y el umbral de separación ($\text{espacio_min} = 10$) son los parámetros que condicionan esta etapa y se determinaron experimentalmente.
- Las letras de la condición final pudieron distinguirse mediante características geométricas simples (cantidad de huecos y trazo izquierdo vertical), lo que resultó suficiente para separar "L" y "R" de las demás.
- El algoritmo se aplicó de forma cíclica a las cuatro planillas, generando por cada una su CSV de validación y, cuando correspondía, la imagen con los alumnos que deben recuperar o están libres.

En conjunto, ambos problemas muestran que técnicas clásicas de procesamiento de imágenes (histogramas locales, umbralización, proyecciones y componentes conectadas) resuelven tareas de análisis de forma automática y reproducible.

ANEXO

Valores máximos de las sumas por columnas y filas

Para cada imagen se listan los 20 mayores valores de las sumas de píxeles oscuros ($\text{img} < 128$) por columna y por fila. El máximo de cada lista es el que se usa en el cálculo del umbral ($\text{th} = 0.8 \cdot \text{máximo}$).

grade_sheet_empty.png (máx. columnas = 520, máx. filas = 788)

Columnas: [103 103 105 105 106 106 107 113 119 129 158 162 443 443 482 482 482 482 483 520]
Filas: [788 788 788 788 788 788 788 788 788 788 788 788 788 788 788 788 788 788 788 788]

grade_sheet_1.png (máx. columnas = 645, máx. filas = 983)

Columnas: [172 174 176 178 182 183 186 187 192 194 205 218 548 548 597 597 597 597 598 645]
Filas: [983 983 983 983 983 983 983 983 983 983 983 983 983 983 983 983 983 983 983 983]

grade_sheet_2.png (máx. columnas = 648, máx. filas = 908)

Columnas: [162 164 164 165 166 168 171 174 177 186 189 548 548 597 597 597 597 636 648]
Filas: [908 908 908 908 908 908 908 908 908 908 908 908 908 908 908 908 908 908 908 908]

grade_sheet_3.png (máx. columnas = 645, máx. filas = 983)

Columnas: [161 163 168 168 172 172 183 188 197 205 205 208 548 548 597 597 597 597 598 645]
Filas: [983 983 983 983 983 983 983 983 983 983 983 983 983 983 983 983 983 983 983 983]

grade_sheet_4.png (máx. columnas = 888, máx. filas = 833)

Columnas: [148 153 154 156 160 169 170 175 176 183 185 204 805 805 848 848 848 848 848 888]
Filas: [833 833 833 833 833 833 833 833 833 833 833 833 833 833 833 833 833 833 833 833]